**Housing Cost and Mental Health Data Pipeline**

## 1. Project Overview

### Project Description

The goal of this project was to design and implement an automated data engineering pipeline that integrates housing, mental health, and homelessness datasets into a centralized PostgreSQL data warehouse. Public datasets from the Centers for Disease Control and Prevention (CDC), the Federal Reserve Economic Data (FRED), and the U.S. Department of Housing and Urban Development (HUD) were extracted, transformed, and loaded into a relational database to support analytical reporting.

The project demonstrates the complete ETL process by automating data extraction, cleaning and transformation, database loading, workflow orchestration with Apache Airflow, and data access through a Flask REST API. Docker Compose was used to containerize the application, allowing all project components to be deployed consistently across different environments.

### Technologies Used

| Technology | Purpose |
|---|---|
| Python | Data extraction, transformation, and API development |
| Pandas | Data cleaning and manipulation |
| PostgreSQL | Relational database |
| SQLAlchemy | Database connectivity |
| Apache Airflow | Workflow automation |
| Papermill | Notebook execution |
| Flask | REST API |
| Docker & Docker Compose | Containerization |
| Jupyter Notebook | ETL development |
| pgAdmin | Database management |

## 2. Data Sources

This project integrates three publicly available government datasets to examine the relationship between housing conditions, public health, and homelessness across the United States. Each dataset contributes a unique perspective and is transformed into a common relational schema within PostgreSQL.

**CDC PLACES**

The CDC PLACES dataset provides county-level health estimates, including measures of poor mental health and insufficient sleep. County FIPS codes and reporting years are used to integrate these health indicators with the housing and homelessness datasets.

**FRED/FHFA Housing Data**

Housing and economic indicators were obtained through the Federal Reserve Economic Data (FRED) service. The project uses the Housing Price Index (HPI), national average rent data, and the 30-year fixed mortgage rate to provide economic context for each reporting year.

**HUD Point-in-Time Homelessness Data**

The U.S. Department of Housing and Urban Development (HUD) publishes annual Point-in-Time (PIT) homelessness estimates by Continuum of Care (CoC). Because CoCs often span multiple counties, a bridge table was created to allocate homelessness estimates to counties using overlap ratios before loading the data into the database.

**Data Integration**

The datasets were standardized using county FIPS codes, state abbreviations, and reporting years. After cleaning and formatting, the three sources were merged into a unified PostgreSQL database, allowing housing, health, and homelessness data to be queried together through a single API.

## 3. Database Design

The project uses a PostgreSQL relational database designed as a **star schema** to efficiently organize and query housing, mental health, and homelessness data. The schema consists of three dimension tables, three fact tables, and one bridge table that resolves the relationship between counties and Continuums of Care (CoCs). This design minimizes data redundancy while supporting efficient analytical queries and API reporting.

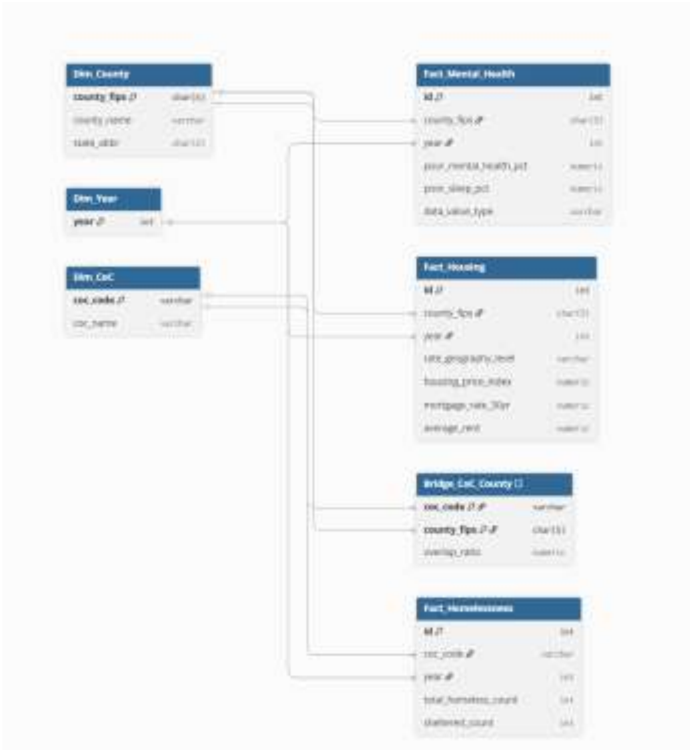


*Figure 1: ERD diagram*

**Dimension Tables**

The dimension tables store descriptive information used throughout the database.

- **Dim_County** contains county names, state abbreviations, and county FIPS codes.
- **Dim_Year** stores the reporting years used across all datasets.
- **Dim_CoC** contains Continuum of Care identifiers and descriptive information used for homelessness reporting.

These tables provide standardized reference data that is linked to the fact tables through foreign keys.

**Bridge Table**

The **Bridge_CoC_County** table resolves the many-to-many relationship between Continuums of Care and counties. Because HUD reports homelessness data by CoC rather than by county, each bridge record contains an overlap ratio that estimates the proportion of a CoC associated with a county. These ratios are used during the transformation process to estimate county-level homelessness values.

**Fact Tables**

The fact tables store the measurable data used for analysis.

- **Fact_Housing** contains housing indicators, including the Housing Price Index, average rent, and mortgage rates.
- **Fact_Mental_Health** stores county-level estimates of poor mental health and insufficient sleep from the CDC PLACES dataset.
- **Fact_Homelessness** contains county-level homelessness estimates generated from the HUD Point-in-Time dataset.

Together, these tables enable the integration of housing, public health, and homelessness data within a single relational database while maintaining referential integrity through foreign key relationships.

**4. ETL Pipeline and Automation**

The ETL pipeline is organized into four Jupyter notebooks, each responsible for a specific stage of the data engineering workflow. Separating the pipeline into modular notebooks improves readability, simplifies debugging, and allows each stage to be executed independently. Apache Airflow orchestrates the workflow by executing each notebook sequentially using Papermill.

**ETL Workflow**

1. **01_extract_cdc_places.ipynb** extracts and prepares county-level mental health and sleep data from the CDC PLACES dataset.
2. **02_extract_fred.ipynb** retrieves housing indicators, including the Housing Price Index, average rent, and 30-year fixed mortgage rate from FRED.
3. **03_extract_HUD.ipynb** processes the HUD Point-in-Time homelessness dataset and prepares the Continuum of Care mapping used during integration.
4. **04_join_data.ipynb** cleans and merges the datasets, allocates homelessness estimates to counties using overlap ratios, and loads the completed dimension, bridge, and fact tables into PostgreSQL.

**Airflow Automation**

The ETL process is automated using Apache Airflow. A custom DAG executes each notebook in the correct order, ensuring that data dependencies are maintained throughout the pipeline.

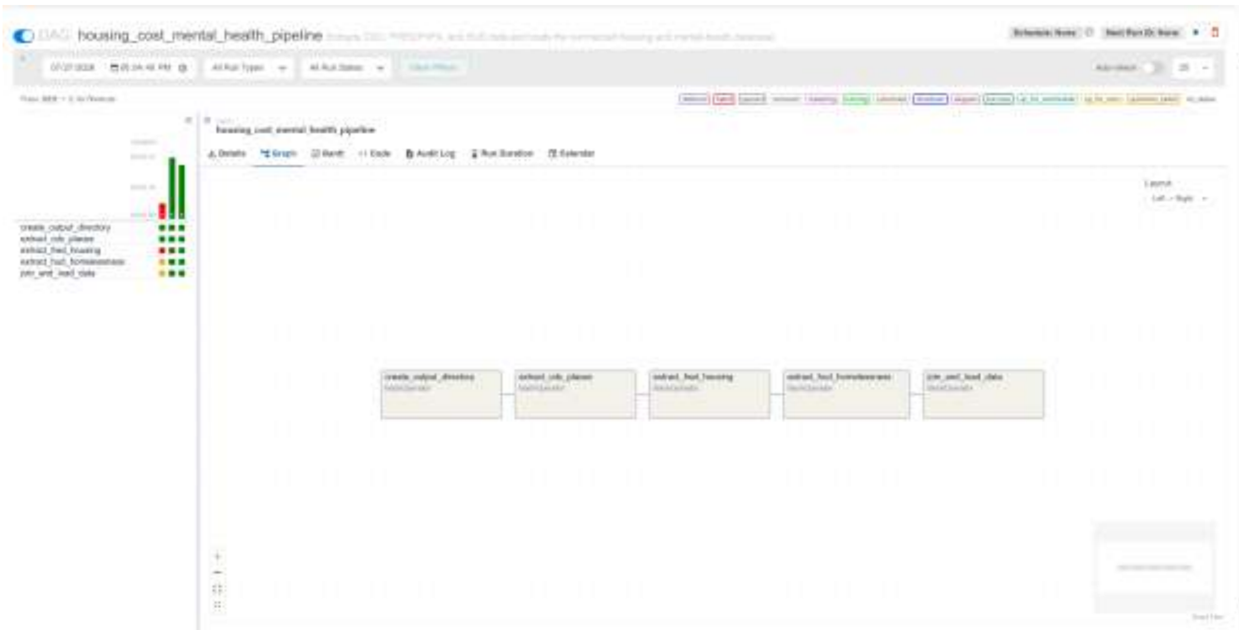


*Figure 2. Apache Airflow DAG executing the complete ETL workflow.*

The execution order is:

create_output_directory

↓

extract_cdc_places

↓

extract_fred_housing

↓

extract_hud_homelessness

↓

join_and_load_data

By automating the workflow with Airflow, the entire pipeline can be executed with a single trigger, making the process reproducible and reducing the need for manual intervention.

**Data Loading**

After the transformation process is complete, the cleaned data is loaded into the PostgreSQL database. The pipeline first populates the dimension tables, followed by the bridge table and fact tables to maintain referential integrity. This loading strategy ensures that all relationships between housing, mental health, and homelessness data are preserved and available for querying through the Flask API.

**5. Flask API and Results**

A Flask REST API was developed to provide access to the data stored in the PostgreSQL database. The API connects to the database using SQLAlchemy and allows users to retrieve aggregated housing, mental health, and homelessness statistics through simple HTTP requests. This provides an interface for future dashboards or analytical applications without requiring direct database access.

**API Endpoints**

The application includes two primary endpoints:

**GET /health**

Verifies that the Flask application can successfully connect to the PostgreSQL database.

Example response:

{

"database": "connected",

"status": "healthy"

}

**GET /api/state-summary**

Returns state-level summary statistics for housing, mental health, and homelessness. Results can be filtered by state and reporting year using optional query parameters.


*Figure 3: Example request: /api/state-summary?state=TX&year=2023*

**Project Results**

The completed pipeline successfully extracts, transforms, and loads data from all three public data sources into the PostgreSQL database. The Airflow workflow automates the ETL process, while the Flask API provides real-time access to aggregated data.

Testing confirmed that the API correctly returned state-level summaries and supported filtering by state and year. For example, querying Texas for 2023 returned aggregated housing, mental health, mortgage rate, and homelessness statistics directly from the database, demonstrating that the ETL pipeline, database schema, and API function together as an integrated system.